

Databázy a vizualizácia

SQL - Príkaz VIEW

doc. Ing. Ladislav Körösi, PhD.

Ústav robotiky a kybernetiky
FEI STU Bratislava

SQL - VIEW

Prečo používame view

View (pohľad) je virtuálna tabuľka vytvorená na základe SQL dotazu.

Databáza neukladá údaje view samostatne, ale ukladá len definíciu dotazu.

Výhody používania view:

- zjednodušenie zložitých dotazov,
- opakované používanie často používaných dotazov,
- obmedzenie prístupu k citlivým údajom.

View sa používa podobne ako bežná tabuľka v príkaze SELECT.

SQL - VIEW

Základný zápis

View vytvoríme pomocou príkazu CREATE VIEW.

```
CREATE VIEW view_name AS  
SELECT ...  
FROM ...  
WHERE ...;
```

Definícia view obsahuje SQL dotaz, ktorý určuje, aké údaje bude view zobrazovať.

View potom môžeme používať rovnako ako tabuľku.

```
SELECT * FROM view_name;
```

Príkaz `CREATE VIEW` umožňuje aj ďalšie nastavenia.

```
CREATE  
[OR REPLACE]  
[ALGORITHM = { UNDEFINED | MERGE | TEMPTABLE }]  
[DEFINER = { user | CURRENT_USER }]  
[SQL SECURITY { DEFINER | INVOKER }]  
VIEW view_name [(column_list)]  
AS select_statement  
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Niektoré z týchto možností si postupne vysvetlíme.

SQL - VIEW

Príklad view

Vytvorme view, ktoré zobrazí názov auta a názov jeho farby.

```
CREATE VIEW auta_farby AS  
SELECT a.nazov AS auto, f.nazov AS farba  
FROM auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id;
```

View potom môžeme používať ako tabuľku.

```
SELECT * FROM auta_farby;
```

alebo napr.

```
SELECT * FROM auta_farby  
WHERE farba = 'biela';
```

SQL - VIEW

CREATE OR REPLACE

Ak view už existuje, môžeme ho jednoducho prepísať.

```
CREATE OR REPLACE VIEW auta_farby AS  
SELECT a.nazov AS auto, f.nazov AS farba  
FROM auta a  
LEFT JOIN farby f  
ON a.farba_id = f.id;
```

Bez použitia OR REPLACE by bolo potrebné najprv view odstrániť pomocou príkazu DROP VIEW.

```
DROP VIEW auta_farby;
```

View môže byť vykonané s rôznymi právami používateľov.

SQL SECURITY DEFINER

- dotaz sa vykoná s právami používateľa, ktorý view vytvoril.

SQL SECURITY INVOKER

- dotaz sa vykoná s právami používateľa, ktorý view používa.

Táto vlastnosť sa využíva napríklad pri riadení prístupov k údajom.

Pri vytváraní view môže MySQL použiť rôzne algoritmy.

MERGE

- dotaz používateľa sa spojí s definíciou view,
- databáza vykoná jeden výsledný dotaz.

TEMTABLE

- databáza najprv vytvorí dočasnú tabuľku z definície view,
- následne vykoná dotaz nad touto tabuľkou.

Algoritmus TEMPTABLE je zvyčajne menej efektívny.

Algoritmus MERGE nie je možné použiť pri všetkých typoch dotazov.

Nie je povolený napríklad ak definícia view obsahuje:

- agregáčné funkcie (SUM, COUNT, AVG, ...),
- DISTINCT,
- GROUP BY alebo HAVING,
- LIMIT,
- UNION,
- vnorené dotazy.

V takýchto prípadoch MySQL automaticky použije algoritmus TEMPTABLE.

View môže obsahovať aj kontrolu vkladaných údajov.

```
CREATE VIEW modre_auta AS  
SELECT *  
FROM auta  
WHERE farba_id = 1  
WITH CHECK OPTION;
```

Podmienka CHECK OPTION zabezpečí, že do view nie je možné vložiť alebo upraviť riadok, ktorý by nespĺňal podmienku definície view. V tomto prípade by nebolo možné vložiť auto, ktorého farba_id nie je rovné 1.

SQL - VIEW

Viac úrovň view

View môže byť definované aj nad iným view.

```
CREATE VIEW modre_auta_nove AS  
SELECT *  
FROM modre_auta  
WHERE id > 2;
```

Takto vznikne reťazec view, kde jedno view využíva výsledok iného view.

LOCAL CHECK OPTION

```
CREATE VIEW modre_auta_nove AS  
SELECT *  
FROM modre_auta  
WHERE id > 2  
WITH LOCAL CHECK OPTION;
```

Kontroluje iba podmienku aktuálneho view.

```
CREATE VIEW modre_auta_nove AS  
SELECT *  
FROM modre_auta  
WHERE id > 2  
WITH CASCADED CHECK OPTION;
```

Kontrolujú sa podmienky všetkých view v reťazci definícií.

V tomto prípade:

- farba_id = 1
- id > 2

SQL - VIEW

Kompletný príklad

```
CREATE OR REPLACE  
ALGORITHM = UNDEFINED  
SQL SECURITY DEFINER  
VIEW auta_farby_view AS  
    SELECT a.id, a.nazov AS auto, f.nazov AS farba FROM auta a  
    LEFT JOIN farby f ON a.farba_id = f.id  
WITH CASCADED CHECK OPTION;
```

- **OR REPLACE** prepíše existujúce view bez nutnosti ho najprv odstrániť,
- **ALGORITHM = UNDEFINED** znamená, že MySQL samo zvolí algoritmus spracovania view. Ak nie je možné použiť **MERGE**, databáza použije **TEMPTABLE**,
- **SQL SECURITY DEFINER** znamená, že dotaz sa vykoná s právami autora view,
- **LEFT JOIN** spojí autá s farbami a zachová všetky autá,
- **CASCADED CHECK OPTION** kontroluje podmienky všetkých view v reťazci.